

Path Tracer Specification

1. Overview

This document specifies a physically based **progressive path tracer** implemented in **C# (.NET 10)** with a later **WPF UI**. The project is explicitly designed as a *learning-oriented, mathematically justified renderer*, developed iteratively with strong documentation and testing.

Key goals:

- Physically correct light transport (rendering equation)
- Progressive, abortable rendering
- Scriptable scenes authored in C#
- Deterministic, testable Monte Carlo rendering
- Extensible architecture (materials, integrators, volumes)

2. Locked Design Decisions

- Language/runtime: **C# (.NET 10)**
- UI: **WPF** (added after core renderer is complete)
- Geometry: spheres + triangle meshes (OBJ initially)
- Color pipeline (early): linear RGB internally, **sRGB + gamma output**
- Color pipeline (later): HDR accumulation + tone mapping
- Preview goal: fast interactive preview (720p preferred, not mandatory)
- Materials before UI: diffuse, mirror, glass, microfacet
- Testing: analytical + statistical + golden-image tests
- Vector math: **custom immutable Vec3**
- Unit testing framework: **xUnit**

3. High-Level Architecture

Projects

- **Tracer.Core**
- Math (Vec3, Ray)
- Random number generation
- Geometry + acceleration (later)

- BSDFs, lights, cameras, integrators
- **Tracer.SceneScripting**
- Typed C# scene authoring API
- Helper geometry and materials
- **Tracer.Renderer**
- Render loop, tiling, multithreading
- Progressive accumulation
- Cancellation support
- **Tracer.UI.Wpf**
- Progressive display
- Abort/start controls
- **Tracer.Tests**
- Unit tests
- Statistical tests
- Golden image regression tests
-

4. Mathematical Foundation

Rendering Equation

For a surface point x and outgoing direction ω_o :

$$L(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} f_r(x, \omega_i, \omega_o) L(x, \omega_i) (\omega_i \cdot n) d\omega_i$$

Monte Carlo Estimator

$$L = L_e + (f_r \cdot L_i \cdot \cos\theta) / pdf$$

- Unbiased Monte Carlo estimation
- Russian roulette termination
- Multiple importance sampling (MIS)

All sampling decisions must provide:

- sampled direction
- PDF
- contribution weight

5. Core Math Types (Milestone 0)

Vec3

- Immutable readonly struct
- float-based
- Dot, cross, normalization
- Near-zero detection
- Hadamard product

Ray

- Origin, direction, time
- Time enables motion blur

6. Deterministic Randomness

RNG

- PCG32 (XSH RR)
- Deterministic given seed
- Suitable for Monte Carlo

Determinism Policy

Randomness is derived from:

(pixelX, pixelY, sampleIndex, baseSeed)

This guarantees:

- Reproducible renders
- Thread-order independence
- Stable golden-image tests

7. Scene Scripting (Target API)

Scenes are authored in C# using a typed builder-style API:

- Camera setup
- Geometry helpers (Sphere, Box, Mesh)
- Materials and lights
- Animation parameters

Example (illustrative):

```
SceneBuilder
```

```
.Camera(...)
```

```
.Add(Box(...))
```

```
.AddLight(RectAreaLight(...))
```

```
.Build();
```

```
---
```

8. Rendering Features

Cameras

- Pinhole
- Thin-lens depth of field
- Motion blur (time sampling)

Geometry

- Sphere (analytic)
- Triangle mesh (OBJ)
- BVH acceleration

Materials

- Lambertian diffuse
- Perfect mirror
- Dielectric glass (Fresnel, TIR)
- Microfacet GGX (conductor + dielectric)

Lights

- Area lights
- (Later) environment maps

9. Progressive Rendering

- Tile-based rendering
- Progressive accumulation buffer
- Partial image always valid
- Abortable via CancellationToken

10. Testing Strategy

Analytical Tests

- Intersection correctness
- Fresnel values at known angles
- PDF conversions
- Transmittance formulas (later volumes)

Statistical Tests

- Sampling distributions (cosine hemisphere, GGX, exponential)
- Monte Carlo estimator convergence checks

Golden Image Tests

- Small fixed-resolution renders
- Fixed seeds
- RMSE/PSNR tolerance comparison

11. Milestones

Milestone 0 — Foundation ■

- Immutable Vec3
- Ray with time
- PCG32 RNG
- Seed hashing

- xUnit test infrastructure

Milestone 1 — Rays, Camera, Spheres

- Pinhole camera
- Ray–sphere intersection
- Image output

Milestone 2 — Diffuse Path Tracing

- Rendering equation
- Monte Carlo integration

Milestone 3 — Emission + Cornell Box

Milestone 4 — Direct Lighting + MIS

Milestone 5 — Meshes + BVH

Milestone 6 — Mirror + Glass

Milestone 7 — Microfacet Materials

Milestone 8 — Thin-Lens DoF

Milestone 9 — Motion Blur

Milestone 10 — WPF UI

Milestone 11 — HDR + Tone Mapping

Milestone 12 — SPPM Integrator (Late Goal)

Milestone 13 — Volumetric Rendering (Late Goal)

- Homogeneous smoke
- Exponential free-flight sampling
- Phase functions (Henyey–Greenstein)

- MIS in participating media

12. Definition of Done (Each Milestone)

- Feature implemented and used by at least one scene
- Mathematical justification documented
- Unit/statistical tests added
- Reference image produced

13. Guiding Principles

- Correctness before optimization
- Determinism over convenience
- Explicit math over magic
- Architecture that teaches